📄 Data Manipulation Language (DML), Transaction Mechanics, and Cybersecurity

---

📑 Table of Contents

---

1. The DML Lifecycle: Create, Read, Update, Delete (CRUD)

Once the Data Definition Language (DDL) establishes the rigid structural blueprint of the database, the **Data Manipulation Language (DML)** is utilized to push data through that structure. In software engineering, this is universally referred to as **CRUD**.

1.1 Create (`INSERT INTO`)

The `INSERT` command writes new leaf nodes into the database's B-Tree.

```
INSERT INTO users (first_name, email) VALUES ('Ada', 'ada@core.com');
```

- *Mechanics:* The engine intercepts the payload, dynamically assigns the `user_id` (via the `AUTOINCREMENT` property), validates the `email` string against the `UNIQUE` constraint, and writes the data to the hard drive.

1.2 Read (`SELECT`)

The `SELECT` command extracts data from storage into RAM.

```
SELECT first_name, email FROM users WHERE user_id = 1;
```

- *Systems Design Rationale:* **Never use `SELECT *` in production.** If a table has 50 columns, `SELECT *` forces the disk controller to read and transmit 50 columns worth of data across the network, even if the application only needs 1 column. Specifying exact columns (`first_name, email`) reduces network payload size and prevents Out-of-Memory (OOM) application crashes.

1.3 Update (`UPDATE`)

Modifies existing states.

```
UPDATE users SET account_balance = 500.0 WHERE user_id = 1;
```

- *Warning:* If an `UPDATE` statement is executed *without* a `WHERE` clause, the database will overwrite the `account_balance` for every single row in the entire table. This is a catastrophic, unrecoverable data-loss event.

1.4 Delete (`DELETE`)

Erases the node from the tree.

```
DELETE FROM users WHERE user_id = 1;
```

- *Alternative (Soft Delete):* In enterprise systems, `DELETE` is rarely used. Hard-deleting data breaks historical audit logs. Instead, engineers use **Soft Deletes**: adding an `is_active` boolean column, and using an `UPDATE` statement to flip it to `0` (False).

---

2. Hardware Transaction Mechanics (The WAL & Commits)

Executing an `INSERT` or `UPDATE` in Python does not immediately change the `.db` file on your hard drive.

2.1 The Rollback Journal & Write-Ahead Log (WAL)

Relational Databases strictly enforce **Atomicity** (all or nothing).
When a Python script begins a DML operation, the database engine writes the intended changes to a temporary hidden cache called the Write-Ahead Log (WAL) or Rollback Journal. If the server's power cord is ripped out of the wall halfway through the query, the database simply discards the temporary cache, ensuring the core `.db` file remains completely uncorrupted.

2.2 The `connection.commit()` Method

The data remains entirely in temporary RAM until explicitly told to finalize.

```
cursor.execute("UPDATE users SET balance = 500 WHERE id = 1")
connection.commit() # 🛑 CRITICAL TRIGGER
```

- *Mechanics:* `commit()` sends an OS-level `fsync()` system call. This orders the solid-state drive (SSD) to flush the RAM buffer and permanently alter the magnetic/silicon state of the hard drive. Without `commit()`, the transaction is inherently lost when the Python script exits.

---

3. State Verification: Telemetry in the Application Layer

Unlike Application code (which throws immediate exceptions if an index is out of bounds), SQL databases fail silently on targeted operations.

If you execute:
`DELETE FROM users WHERE user_id = 999;`
And User 999 does not exist, the SQL engine returns a `SUCCESS` status. From the database's perspective, it successfully completed the operation by deleting exactly zero rows.

The Engineering Fix: `cursor.rowcount`
To provide state telemetry back to the backend application, we utilize the `rowcount` variable.

```
cursor.execute("UPDATE users SET balance = 0 WHERE id = 999")
connection.commit()

if cursor.rowcount == 0:
    # Trigger 404 HTTP Response back to the frontend
    print("Error: Targeted user does not exist in the dataset.")
```

This telemetry ensures the Python runtime handles missing data dynamically, rather than falsely confirming successful operations to the end-user.

---

4. Threat Modeling: The Anatomy of SQL Injection (SQLi)

SQL Injection (SQLi) is consistently ranked in the OWASP Top 10 most critical security risks in the world. It occurs when untrusted user input is concatenated directly into a database query string.

4.1 The Vulnerability Mechanics (String Interpolation)

Consider a vulnerable Python backend:

```
# 🚨 CATASTROPHIC SECURITY FLAW
user_input = "Jainish"
query = f"SELECT * FROM users WHERE first_name = '{user_input}';"
cursor.execute(query)
```

If a malicious actor inputs the string: `Jainish'; DROP TABLE users; --`
The Python interpreter blindly evaluates the f-string, yielding:

```
SELECT * FROM users WHERE first_name = 'Jainish'; DROP TABLE users; --';
```

- The semi-colon `;` terminates the first query.
- The `DROP TABLE` command deletes the entire user database.
- The `--` comments out the trailing quote, preventing syntax errors.
  The database executes the payload exactly as instructed, destroying the company.

---

5. Cyber Defense: Parameterized Queries & Compiler Theory

To neutralize SQLi, enterprise engineering bans string formatting (`f-strings` , `.format()` , `%s`) in SQL operations.
The absolute defense is **Parameterized Query Binding**.

5.1 The Implementation (`?` Placeholders)

```
# 🛡️ ENTERPRISE SECURE CODE
user_input = "Jainish'; DROP TABLE users; --"
query = "SELECT * FROM users WHERE first_name = ?;"

# Note the Tuple requirement: (user_input,)
cursor.execute(query, (user_input,))
```

5.2 Deep Mechanics: Compiler Decoupling

Why does this block the attack?

1. **Phase 1 (Compilation):** The Python driver sends the SQL string (`SELECT * FROM users WHERE first_name = ?`) to the database engine first, completely alone. The engine compiles this string into a rigid Abstract Syntax Tree (AST) in memory.
2. **Phase 2 (Binding):** The driver then sends the malicious user input.
3. **Phase 3 (Sanitization):** Because the AST is already rigidly compiled, the database refuses to alter the structure. It takes the malicious input and forcibly binds it as a scalar literal value.

The database physically searches the hard drive for a user whose literal first name is `"Jainish'; DROP TABLE users; --"` . It finds zero matches, returns safely, and the attack is fundamentally neutralized.

---

6. Advanced DML: Upserts and Pipeline Idempotency

In modern Data Engineering, ELT (Extract, Load, Transform) pipelines run via CRON jobs asynchronously (e.g., every 5 minutes). A pipeline must be **Idempotent**—it must be able to

run 10,000 times without crashing or mutating the database beyond the intended state.

If a script blindly executes `INSERT INTO` on data that already exists, it triggers a `UNIQUE` constraint violation and crashes the entire pipeline.

6.1 The "Upsert" (Update or Insert)

We utilize native conflict-resolution syntax.

```
-- SQLite 'INSERT OR IGNORE'
INSERT OR IGNORE INTO users (email) VALUES ('ada@core.com');

-- Enterprise PostgreSQL 'ON CONFLICT'
INSERT INTO users (email) VALUES ('ada@core.com')
ON CONFLICT (email) DO NOTHING;
```

- *Mechanics:* The engine intercepts the `INSERT`. If the secondary index detects that `ada@core.com` already exists, it suppresses the `IntegrityError`, abandons the write-lock, and seamlessly moves to the next row in the batch payload. This guarantees perfect pipeline idempotency and continuous server uptime.